A1 Exam Review: LLM Agent Threat Modelling

Deduplicated from materials/A1-solution.md

2026-06-23

Contents

What To Reuse From A1

This sheet extracts the non-repeated examable structure from `materials/A1-solution.md`. It is not a resubmission of A1. Use it as a compact answer template for exam questions about architecture diagrams, trust boundaries, STRIDE, impacts, and mitigations.

Core exam schema:

1. Identify components and who controls them.
2. Group components into trust domains by control, ownership, hosting, and assumptions.
3. Label legitimate communication channels and sensitive information.
4. Apply STRIDE to assets, channels, and trust-boundary crossings.
5. State impact in terms of who is harmed and which security property fails.
6. Write requirements as concrete system controls, not generic advice.

Architecture Snapshot

Component	Controller	Exam role
User Web Browser	User	Web sign-up, login, account management.

Component	Controller	Exam role
CLI Application	User	Local interface to the AI agent; authenticates, restores/saves sessions, sends prompts and tool results to LLM.
Public Web Interface	Company app, CDN infrastructure	Account/session management, saved histories, usage records, billing-facing web functions.
Remote LLM Service	Company service, cloud GPU infrastructure	Runs model inference; processes prompts, conversation history, and tool outputs; may be retrained.
Identity Provider	Third party	Authenticates website and CLI users.
Billing Provider	Third party	Stores billing details and calculates monthly bills from usage.
Administrator Computers / Internal Network	Company	Management plane for public web and LLM service, including maintenance and retraining.

Exam rule: a component can have split operational control and infrastructure control. Say this explicitly when the app is company-controlled but hosted by a CDN or cloud provider.

Trust Boundaries

Boundary	Contains	Why it is separate
User-controlled endpoint	Browser, CLI	Runs on the user's machine, outside company/provider control.
CDN-hosted public web	Public Web Interface plus its session/usage database	Public-facing app in a distinct hosting environment.
Cloud-hosted LLM	Remote LLM Service	Separate cloud GPU infrastructure and model-processing environment.
Identity provider	Identity Provider	External authentication authority trusted but not controlled by the company.
Billing provider	Billing Provider	External billing authority storing financial data.
Internal administration	Admin computers/internal network	Privileged management plane, separate from public and user environments.

Boundary rule: draw boundaries around trust domains, not around individual features. Do not pass a boundary through a component.

Communication And Sensitive Data

Channel	Purpose	Sensitive information
Browser <-> Public Web	Sign-up, login, account management	Credentials, account data, billing data during sign-up, website session state.
Public Web <-> IdP	Website authentication	Authentication tokens, assertions, session credentials.

Channel	Purpose	Sensitive information
CLI <-> IdP	CLI authentication	Authentication tokens, assertions, session credentials.
CLI <-> Public Web	Save/restore sessions, autosave histories, usage tracking	Session histories, prompts/outputs, session names, usage records, account identifiers.
CLI <-> Remote LLM	Model inference	User prompts, model responses, conversation history, tool outputs.
Public Web <-> Billing	Billing submission and monthly usage exchange	Billing details, paid-user usage, account identifiers.
Admin Network <-> Public Web	Administration and maintenance	Admin credentials, management commands, privileged access to sessions/usage.
Admin Network <-> Remote LLM	Maintenance and retraining	Admin credentials, model configuration, retraining data derived from feedback.

Storage to remember:

- Public Web stores session data, histories, saved CLI sessions, usage/query records, and website session state.
- Billing Provider stores customer billing details and billing records.
- Identity Provider stores authentication material.
- Admin environment stores or handles privileged management material.
- Do not claim persistent storage by Browser, CLI, or LLM unless the scenario states it.

Assumptions Worth Stating

- Successful authentication produces tokens, assertions, or equivalent credentials.
- Website session management implies some server-side session state.
- Admin channels carry privileged authentication and management traffic.
- Tool outputs may be appended to model conversation context.
- Usage data must be linkable to accounts for billing.
- Feedback used for retraining becomes part of the model-improvement pipeline.

Exam rule: assumptions should be narrow, necessary, and tied to the scenario. Avoid inventing broad extra system features.

STRIDE Trigger Table

STRIDE	Property threatened	In this A1 scenario, look for…
Spoofing	Authentication	Fake LLM backend, fake Codey/IdP login, stolen CLI token reused as the user.
Tampering	Integrity	Modified saved sessions, forged usage updates, tampered tool output, poisoned feedback.
Repudiation	Accountability	User denies tool action, paid user disputes usage, admin denies privileged change.
Information disclosure	Confidentiality	Exposed saved histories, intercepted CLI-to-LLM traffic, cloud insider reads processed data.
Denial of service	Availability	Free-tier Sybil abuse, infinite agent tool loop, flooding account/session endpoints.

STRIDE	Property threatened	In this A1 scenario, look for…
Elevation of privilege	Authorization	Prompt injection causes local tool execution, one user accesses another user's sessions, admin endpoint compromise, free user bypasses entitlement.

Fast classification checks:

- Fake identity or fake service: Spoofing.
- Changed data, history, output, usage, or feedback: Tampering.
- Cannot prove who did what: Repudiation.
- Unwanted reading or exposure: Information Disclosure.
- Legitimate users cannot use the system: Denial of Service.
- Actor gains abilities beyond their role: Elevation of Privilege.

Threats, Impacts, Controls

Use this as a compact replacement for the repeated threat-impact-mitigation prose.

Threat pattern	Main impact	Matching control pattern
Spoofed LLM backend returns malicious tool requests	Local file deletion, data exfiltration, unwanted command execution, loss of trust in the agent workflow.	Verify LLM server identity with TLS/certificate/hostname validation; reject unauthenticated endpoints; require confirmation for risky tools.
Fake Codey login or IdP consent flow	Account takeover, saved-session exposure, billing/account misuse.	Pre-register IdP endpoints and redirect targets; use phishing-resistant MFA; bind auth responses to legitimate service origins.
Stolen CLI auth token	Attacker acts as user without password; saved sessions and paid usage are exposed.	Store tokens in OS credential store, use short-lived tokens, support revocation and re-authentication.
Tampered saved session history	Restored context misleads model, corrupts later reasoning, injects malicious instructions.	Treat restored sessions as untrusted input; version and validate changes; keep tamper-evident logs.
Manipulated client-side usage reporting	Underbilling, overbilling, unfair quota enforcement, billing disputes.	Make Public Web authoritative for usage; validate server-side; log usage updates.
Tampered tool output or prompt-injected content	Model acts on attacker-controlled instructions or false data.	Separate tool data from system instructions; filter meta-instructions; require approval for sensitive tool actions.
Feedback poisoning for retraining	Future model quality/safety degrades for many users.	Link feedback to authenticated sessions, rate-limit, detect anomalies, review before retraining.
User denies harmful tool invocation	Dispute over whether user, model, or attacker caused the action.	Append-only logs linking prompt, model tool request, user confirmation, execution result, account, and session.
Paid user disputes query volume	Revenue loss and support burden.	Tamper-evident usage logs tied to authenticated sessions.
Admin denies privileged change	Weak incident response and insider accountability.	Individually assigned admin accounts; immutable per-admin logs; protect logs from the same admin.

Threat pattern	Main impact	Matching control pattern
Web-interface compromise exposes saved histories	Confidential source code, secrets, personal data, or conversations leak.	Least-privilege access, encryption at rest, strict admin/user separation, object-level authorization.
CLI-to-LLM interception	Prompts, responses, and tool outputs leak in transit.	Strong transport protection and server identity verification; data minimization and redaction.
Cloud-hosting personnel inspect LLM data	Confidentiality and compliance risk despite company not intending misuse.	Minimize sent data, limit retention, isolate roles, least privilege across hosted components.
Sybil free-tier abuse	LLM capacity/cost exhaustion; legitimate users lose access.	Anti-automation at sign-up, per-account and per-origin limits, server-side entitlement checks.
Excessive agent loop	Local and backend resources tied up; expensive non-terminating sessions.	Tool-iteration limits, compute budgets, timeouts, safe termination conditions.
Flooding session/account endpoints	Login, save, restore, and billing-related functions fail.	Rate-limit stateful endpoints, prioritize critical flows, isolate backend storage paths.
Untrusted content triggers local tool execution	External content gains influence over local privileged actions.	Treat fetched/file content as data, not instructions; require explicit confirmation for high-risk tools.
Cross-user saved-session access	One customer reads, modifies, or deletes another customer's histories.	Non-predictable identifiers, ownership checks on every restore/update/delete, server-side authorization.
Admin endpoint compromise	System-wide control over sessions, usage, model config, and retraining.	Dedicated admin workstations, MFA, network segmentation, least privilege, separation of duties, dual control for high-impact changes.
Free user bypasses web-layer controls	Unpaid backend model use and degraded service for others.	Enforce entitlement on the service side, not only in client-visible workflow.

Requirement Writing Template

Use this structure in exam answers:

`The system should [specific control] for [asset/channel/component] so that [threat/failure] is prevented, detected, or limited.`

Good examples:

- The CLI should verify the Remote LLM Service identity on every connection and reject invalid certificates, so that spoofed model endpoints cannot return malicious tool invocations.
- The Public Web Interface should check saved-session ownership on every restore, update, and delete request, so that one authenticated user cannot access another user's histories.
- The agent runtime should enforce maximum tool-iteration counts and tool timeouts, so that prompt injection or bad model behaviour cannot create unbounded loops.

Weak examples:

- "Use encryption" without saying what data or channel it protects.
- "Improve security" without naming a threat.
- "Add logging" without making logs attributable, tamper-evident, and linked to sessions/actions.

Common Exam Mistakes

1. Naming generic threats not tied to the architecture.
   - Weak: “Hackers steal data.”
   - Strong: “A compromise of the Public Web Interface exposes centrally stored saved session histories.”
2. Confusing STRIDE categories.
   - Phishing or fake service identity usually maps to Spoofing.
   - Modifying saved history, tool output, usage, or feedback maps to Tampering.
   - Cross-user session access usually maps to Elevation of Privilege because an authenticated user gains unauthorized capability.
3. Treating trust boundaries as feature boxes.
   - Boundaries follow control and trust assumptions: user endpoint, public web/CDN, cloud LLM, IdP, billing provider, admin network.
4. Forgetting the LLM-agent-specific data path.
   - Tool output and restored sessions become model context, so they can affect later tool use.
5. Writing controls at the wrong layer.
   - Free-tier limits and usage integrity must be enforced server-side, not only by the CLI.
6. Overclaiming storage.
   - Only claim stored sensitive data where the assignment states it or where a narrow assumption is justified.

Mini Self-Test

1. A user restores a saved session, but the stored history contains attacker-written instructions that later influence tool use. Which STRIDE category is the cleanest fit, and why?
2. A free-tier user scripts direct model requests that bypass the Public Web Interface’s visible quota checks. Is this mainly DoS, Elevation of Privilege, or both? State the exam reason.
3. Why is “encrypt everything” not a full mitigation for cross-user saved-session access?
4. In one sentence, explain why the CLI-to-LLM channel is more sensitive than a simple API request channel.
5. What must an audit log include to help with repudiation of a harmful tool invocation?

Answer targets:

1. Tampering, because saved context is modified and then corrupts later model reasoning/tool behavior.
2. Elevation of Privilege for unauthorized service entitlement; it may also cause DoS if scaled to exhaust capacity.
3. Because authorization still requires ownership checks on every restore/update/delete; encryption alone does not prove requester rights.
4. It may contain prompts, model responses, conversation history, and tool outputs such as local file contents.
5. Authenticated account, session, prompt/request, model tool request, user confirmation if required, execution outcome, timestamp, and tamper-evidence.

One-Page Recall Map

Architecture:

`User endpoint -> Public Web -> IdP/Billing -> LLM cloud -> Admin network`

High-value assets:

`saved sessions`, `usage records`, `auth tokens`, `billing data`, `tool outputs`, `feedback/retraining data`, `admin credentials`

STRIDE memory:

`S = fake identity`, `T = changed data`, `R = denial without evidence`, `I = exposed data`, `D = unavailable service`, `E = unauthorized capability`

LLM-agent special risks:

`tool invocation`, `prompt injection`, `restored context`, `tool-output contamination`, `feedback poisoning`, `runaway loops`